Access Control and Authentication Assignment

1. Mock Zomato Admin Login Form

admin_user = "admin" admin_pass = "zomato123" username = input("Username: ") password = input("Password: ") if username == admin_user and password == admin_pass: print("Access Granted") else: print("Access Denied")

2. Role-Based Access Control Function

def check_access(user_role, resource): permissions = { "customer": ["order_history"], "restaurant_owner": ["menu_edit", "order_history"], "admin": ["menu_edit", "order_history", "admin_dashboard"] } return resource in permissions.get(user_role, []) print(check_access("customer", "order_history")) print(check_access("restaurant_owner", "menu_edit")) print(check_access("customer", "admin_dashboard"))

3. Two-Factor Authentication (OTP)

import random password = input("Enter Password: ") if password == "paytm123": otp = random.randint(100000, 999999) print("OTP:", otp) user_otp = int(input("Enter OTP: ")) if user_otp == otp: print("Login Successful")

4. Filter Premium Users Logged In Within 7 Days

from datetime import datetime def filter_users(users): today = datetime.strptime("2024-06-10", "%Y-%m-%d") return [u for u in users if u["isPremium"] and (today - datetime.strptime(u["lastLogin"], "%Y-%m-%d")).days <= 7]

5. Common Access Control Mistakes

1. Missing authorization checks - Verify permissions on every request. 2. Excessive user privileges - Apply least-privilege access. 3. Insecure admin endpoints - Protect admin routes with RBAC. 4. Predictable session tokens - Use strong cryptographic tokens. 5. Weak password policies - Enforce strong passwords and MFA.